

OBJECT ORIENTED PROGRAMMING [CT-260]



NAME: ABDUL RAHEEM SHEIKH

ROLL NO: CT-25192

DEPARTMENT: BCIT

BATCH: 2025

YEAR & SECTION: FSCS-D

LAB: 09

CODES

QUESTION 01

```
#include <iostream>
using namespace std;

// Abstract Class
class ArrayMultiplier {
public:
    virtual void calculate() = 0;
};

// 1D Array Multiplication
class ArrayMultiplier1D : public ArrayMultiplier {
    int arr[5];

public:
    ArrayMultiplier1D() {
        cout << "Enter 5 elements for 1D array:\n";
        for (int i = 0; i < 5; i++) {
            cin >> arr[i];
        }
    }

    void calculate() {
        int product = 1;

        for (int i = 0; i < 5; i++) {
            product *= arr[i];
        }

        cout << "1D Array Multiplication = " << product << endl;
    }
};

// 2D Array Multiplication
class ArrayMultiplier2D : public ArrayMultiplier {
    int arr[2][2];

public:
    ArrayMultiplier2D() {
        cout << "Enter 4 elements for 2D array:\n";
```

```

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                cin >> arr[i][j];
            }
        }
    }

    void calculate() {
        int product = 1;

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                product *= arr[i][j];
            }
        }

        cout << "2D Array Multiplication = " << product << endl;
    }
};

int main() {

    ArrayMultiplier *ptr;

    ArrayMultiplier1D obj1;
    ptr = &obj1;
    ptr->calculate();

    ArrayMultiplier2D obj2;
    ptr = &obj2;
    ptr->calculate();

    return 0;
}

```

OUTPUT:

```
Enter 5 elements for 1D array:
6
7
8
9
9
1D Array Multiplication = 27216
Enter 4 elements for 2D array:
5
67
1
2
2D Array Multiplication = 670
```

QUESTION 02

```
#include <iostream>
using namespace std;

// Abstract Class
class Store {
protected:
    float total_bill;

public:
    Store(float bill) {
        total_bill = bill;
    }

    virtual void calculateBill() = 0;
};

// Imtiaz Store
class ImtiazStore : public Store {
public:
    ImtiazStore(float bill) : Store(bill) {}

    void calculateBill() {
        float finalBill = total_bill - (total_bill * 0.07);

        cout << "Imtiaz Store Final Bill = " << finalBill << endl;
    }
}
```

```

};

// Bin Hashim Store
class BinHashimStore : public Store {
public:
    BinHashimStore(float bill) : Store(bill) {}

    void calculateBill() {
        float finalBill = total_bill - (total_bill * 0.05);

        cout << "Bin Hashim Store Final Bill = " << finalBill << endl;
    }
};

int main() {

    float bill;

    cout << "Enter Total Bill: ";
    cin >> bill;

    Store *ptr;

    ImtiazStore i(bill);
    ptr = &i;
    ptr->calculateBill();

    BinHashimStore b(bill);
    ptr = &b;
    ptr->calculateBill();

    return 0;
}

```

OUTPUT:

```

Enter Total Bill: 45000
Imtiaz Store Final Bill = 41850
Bin Hashim Store Final Bill = 42750

```

QUESTION 03

```
#include <iostream>
#include <string>
using namespace std;

// Abstract Base Class
class Vehicle {
protected:
    int carId;
    string brand;
    string model;

public:
    Vehicle(int id, string b, string m) {
        carId = id;
        brand = b;
        model = m;
    }

    virtual bool isAvailable() = 0;
    virtual void rent() = 0;
    virtual void returnVehicle() = 0;

    virtual void display() {
        cout << "Car ID: " << carId << endl;
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
    }

    virtual ~Vehicle() {}
};

// Derived Class
class Car : public Vehicle {
    bool available;

public:
    Car(int id, string b, string m)
        : Vehicle(id, b, m) {
        available = true;
    }
}
```

```

    bool isAvailable() {
        return available;
    }

    void rent() {
        if (available) {
            available = false;
            cout << model << " rented successfully.\n";
        }
        else {
            cout << model << " is already rented.\n";
        }
    }

    void returnVehicle() {
        available = true;
        cout << model << " returned successfully.\n";
    }
};

// Rental System
class RentalSystem {
public:
    void rentVehicle(Vehicle *v) {

        if (v->isAvailable())
            v->rent();
        else
            cout << "Vehicle not available.\n";
    }

    void returnVehicle(Vehicle *v) {
        v->returnVehicle();
    }
};

// Customer Class
class Customer {
    string name;

public:
    Customer(string n) {
        name = n;
    }
};

```

```

void rentVehicle(RentalSystem &system, Vehicle *v) {
    cout << name << " wants to rent vehicle.\n";
    system.rentVehicle(v);
}

void returnVehicle(RentalSystem &system, Vehicle *v) {
    cout << name << " returned vehicle.\n";
    system.returnVehicle(v);
}
};

int main() {

    Vehicle *vehicles[2];

    vehicles[0] = new Car(1, "Toyota", "Corolla");
    vehicles[1] = new Car(2, "Honda", "Civic");

    RentalSystem system;

    Customer c1("Ali");

    c1.rentVehicle(system, vehicles[0]);

    cout << endl;

    c1.returnVehicle(system, vehicles[0]);

    // Free memory
    for (int i = 0; i < 2; i++) {
        delete vehicles[i];
    }

    return 0;
}

```

OUTPUT:

Ali wants to rent vehicle.
Corolla rented successfully.

Ali returned vehicle.
Corolla returned successfully.

QUESTION 04

```
#include <iostream>
#include <string>
using namespace std;

// Abstract Class
class EncryptionTechnique {
public:
    virtual void encrypt(string message) = 0;
};

// Technique 1
class EncryptionTechnique1 : public EncryptionTechnique {
public:
    void encrypt(string message) {

        string result = "";

        for (int i = 0; i < message.length(); i++) {
            result += to_string((int)message[i]);
        }

        cout << "Technique 1 Encryption: " << result << endl;
    }
};

// Technique 2
class EncryptionTechnique2 : public EncryptionTechnique {
public:
    void encrypt(string message) {

        string result = "";

        for (int i = 0; i < message.length(); i++) {
            result += to_string((int)message[i] + 2);
        }

        cout << "Technique 2 Encryption: " << result << endl;
    }
};
```

```

    }
};

int main() {

    string message;

    cout << "Enter Message: ";
    getline(cin, message);

    EncryptionTechnique *ptr;

    EncryptionTechnique1 e1;
    ptr = &e1;
    ptr->encrypt(message);

    EncryptionTechnique2 e2;
    ptr = &e2;
    ptr->encrypt(message);

    return 0;
}

```

OUTPUT:

```

Enter Message: hello
Technique 1 Encryption: 104101108108111
Technique 2 Encryption: 106103110110113

```

QUESTION 05

```

#include <iostream>
#include <string>
using namespace std;

// Abstract Class
class DecryptionTechnique {
public:
    virtual void decrypt(string code) = 0;
};

```

```

// Technique 1 Decryption
class DecryptionTechnique1 : public DecryptionTechnique {
public:
    void decrypt(string code) {

        cout << "Technique 1 Decryption: ";

        for (int i = 0; i < code.length(); i += 2) {

            string temp = code.substr(i, 2);

            char ch = stoi(temp);

            cout << ch;

        }

        cout << endl;
    }
};

// Technique 2 Decryption
class DecryptionTechnique2 : public DecryptionTechnique {
public:
    void decrypt(string code) {

        cout << "Technique 2 Decryption: ";

        for (int i = 0; i < code.length(); i += 2) {

            string temp = code.substr(i, 2);

            char ch = stoi(temp) - 2;

            cout << ch;

        }

        cout << endl;
    }
};

int main() {

    string code1, code2;

    cout << "Enter Technique 1 Encrypted Code: ";

```

```
cin >> code1;

cout << "Enter Technique 2 Encrypted Code: ";
cin >> code2;

DecryptionTechnique *ptr;

DecryptionTechnique1 d1;
ptr = &d1;
ptr->decrypt(code1);

DecryptionTechnique2 d2;
ptr = &d2;
ptr->decrypt(code2);

return 0;
}
```

OUTPUT:

```
Enter Technique 1 Encrypted Code: 7269767679
Enter Technique 2 Encrypted Code: 7471787881
Technique 1 Decryption: HELLO
Technique 2 Decryption: HELLO
```